

Unfortunately time is not on the menu.

1. Project Overview

The game revolves around trying to survive while balancing a minigame and the constant harassment from the threat. You must protect yourself according to the characteristics of that threat. Which include closing the door, turning the light off. wearing a mask and closing the pc. The difficulty will increase exponentially by time until the player score reaches 100 by doing minigames.

2. Project Review

This game takes inspiration from five nights at freaddy which will be referred to as fnaf. Some mechanics are taken from fnaf 1, fnaf 2 except the camera system. Due to the reason of how useless they are and little to no gameplay value can be extracted from them. While the original pace is slower and tries to build the horror atmosphere to spook the player, with this game we took a different approach which is focusing on multitasking where everything is coming at the same time. Forcing players to make split decisions or fail.

3. Programming Development

3.1 Game Concept

The player is trapped inside an office room containing a door, a vent, and a window that allows them to look outside. The objective is to survive by playing a minigame until it reaches a certain point, all while preventing threats from entering the room or killing the player. Different threats may require different actions to defend against them. For example, the player may need to close the door, turn off the lights, or wear a mask, depending on the type of threat approaching.

3.2 Object-Oriented Programming Implementation

Animatronic Controller

Purpose: used to manage different animatronic, for example what state they need to be, whatever the sprite should be drawn in this frame etc.

Attribute:

1. **Animatronics** - A list of all the animatronic
2. **States** - A dictionary for storing state for each animatronic
3. **Event handler** - Object from event handler class, used for checking condition.

Key methods:

1. **Process** - basically determined which animatronic is able to advance the state and receive events from the animatronics.
2. **Render** - determine which animatronic should be rendered according to the scene player is on.
3. **Internal Process** - will be used to dissect events received from process and act accordingly. Such as Attack and Reset which will need to be checked with the event handler.

Animatronic System

Purpose: A blueprint for the animatronic. Which will include common behaviour/function and attributes, this include function related to moving, animation, freeze.

Attribute:

1. **Aggro Rate** - Determines how frequently the animatronic changes its state or becomes more aggressive.
2. **State** - Determines which state the animatronic is on (idle, moving, state). Also which used to display sprite in different states.

Key methods:

1. **Try to move** - will randomly select a number from one to ten and check whatever or not it exceeds 100 - Aggro. If it does, the state increases by one. Else repeat till frozen.

2. **Run animation** - Mostly used for animating sprites, which could be played in reverse and return True when the animation is done.
3. **Attack** - Return attack event with class name for Animatonic Controller to process.

Office controller

Purpose: A control for sprite/animation representing the background, this includes the office itself, door, pc.

Attribute:

1. **Office** - Class that stores sprite for the office, which will have a render method.
2. **Door** - Class that stores sprite for door, include how it animate and how it should be rendered.
3. **Back room** - Class that stores sprite for the back room (When player turns to right) which will include the render method.
4. **Mini Game** - Class that will include the logic, sprites for a min game which will include behavior method and render method.
5. **Event handler** - Object from event handler class, used for checking condition.

Key methods:

1. **Process** - a method used for checking condition from event handler and update accordingly.
2. **Render** - a method to render sprite according to the condition.

Event Handler

Purpose: Mostly used to switch between conditions. Such as toggle the light, door, pc etc. Also used to communicate between classes.

Attribute:

1. **Is mask on** - self explanatory
2. **Is light on** - self explanatory
3. **Is pc on** - self explanatory
4. **Is door close** - self explanatory

5. Is facing office - self explanatory

Key methods:

1. **Try kill Player** - used to check if animatronic that pass as parameter could kill the player or not if yes, display jump scare, reset, etc. Else reset the animatronic.

Animation

Purpose: For loading sprite, annotated sprite, and animate the sprite.

Attribute:

1. **frame** - current frame.
2. **sprites** - a list of images/sprites.
3. **Sprite width, Sprite height** - used to determine how large the sprite in the spritesheet so it could be extracted correctly.
4. **Output width, Output height** - How large the sprite draw on the screen should be.

Key methods:

1. **load Sprites** - extract sprite form spritesheet and return it as a list containing images.
2. **Extract Sprite** - extract one sprite from the spritesheet according to the parameter such as the first sprite with a 2px gap.
3. **Animated** - use for updating frames which will be used to render pictures according to that frame.

3.3 Algorithms Involved

The game uses a random number based decision system to control the movement of the animatronics. Every second, the system generates a random integer between 1 and 10. The animatronic will move only if the generated number satisfies the condition $\text{randomInt}(1,10) \geq (100 - \text{aggro})$, where aggro represents the animatronic's aggression level. If the condition is met, the animatronic advances to the next state (such as moving closer to the player). If the condition fails, the animatronic remains in its current state but its aggro value increases slightly, making it more likely to move in future checks. This algorithm creates dynamic behavior where enemies gradually become more aggressive over time, increasing the difficulty as the game progresses.

4. Statistical Data (Prop Stats)

4.1 Data Features

Feature	Why is it useful?	How to get 100 rows?	Variable and Class	Display
---------	-------------------	----------------------	--------------------	---------

Success Rate per Threat Type	How well player handles each enemy	Check if player survived or not	Undecided	Table (Threat vs Success Rate)
Time Survived vs Number of Inputs	Shows if panicked players (high inputs) die earlier	undecided	Undecided	Scatter Plot (Time Survived vs Number of Inputs)
Reaction time according to player score	Shows reaction time trends, revealing panic, learning.	Undecided	Undecided	Line Graph (score vs Reaction Time)
Input Burst Count (per encounter)	Measures how panic a player is during threats.	Undecided	Undecided	Box Plot (Input Burst Count Distribution)
Action Type Usage	Reveals player habits and to find if any mechanics are ignored or overused	Count every interaction done during the game.	Undecided	Bar Graph (Action Type vs Usage Count)

4.2 Data Recording Method

The game records statistical data into a CSV file using an event-based recording system. Every in-game event whether a threat appears, the player reacts, or the night ends is captured as a single row with a timestamp.

CSV Columns (header)

Timestamp - the exact moment the event happened.

Event type - basically type of event (Attack, Door close).

Threat Name - which animatronic triggered the event.

Aggro level - the threat aggro value at the moment.

Action - the action done by the player (Door close, light off).

Survived - whether the player survived the encounter or not.

Score - the player's current score at the time of the event.

Input Count - total inputs made by the player during the encounter.

session duration - total real time since the session started.

4.3 Data Analysis Report

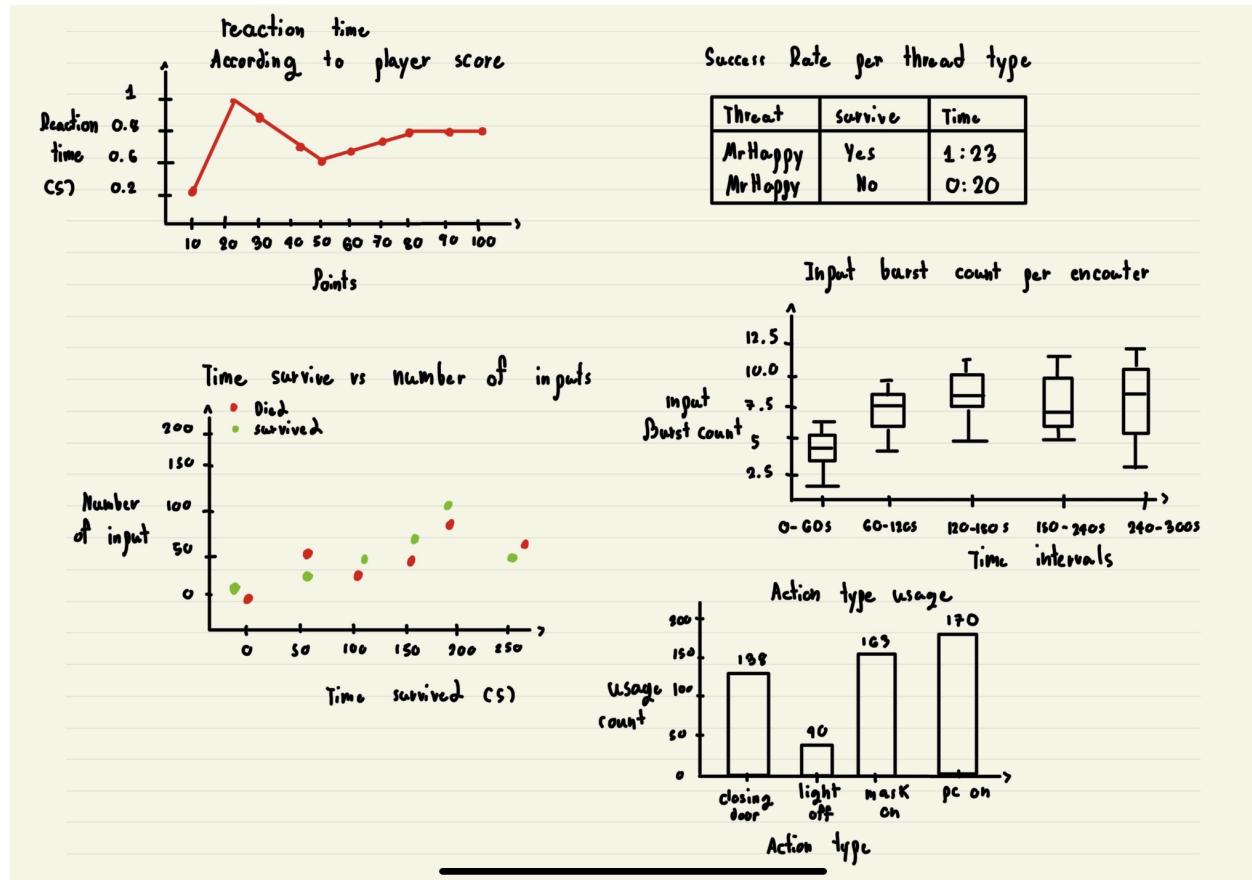
The recorded gameplay data will be analyzed using statistics and data visualization.

The analysis focuses on player behavior, performance trends, and game balance across increasing difficulty.

Feature	What does the graph show and why?	Visualize method	x-axis	y-axis
---------	-----------------------------------	------------------	--------	--------

Success Rate per Threat Type	How well player handles each enemy	Table	Threat	Success Rate
Time Survived vs Number of Inputs	Shows if panicked players (high inputs) die earlier	Scatter Plot	Time Survived	Number of Inputs
Reaction time according to player score	Shows reaction time trends, revealing panic, learning.	Line Graph	score	Reaction Time
Input Burst Count (per encounter)	Measures how panic a player is during threats.	Box Plot	Time interval	Input Burst
Action Type Usage	Reveals player habits and to find if any mechanics are ignored or overused	Bar Graph	Action Type	Usage Count

Hand Drawn Expected Graph



5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Work on gameplay implementation
3	10	Playtesting and feedback
4	11	Work on game assets
5	12	Playtesting and feedback

6	13	Work on statistics and data analysis
7	14	Submission week (Draft)

6. Document version

Version: 1.1

Date: 28 March 2025

Editor: Pathadon Aougsk 6810545808

Date	Name	Description of Revision, Feedback, Comments
14/3	Nattanan	1. this might lead to you playing 100 night, make sure the length of the game is suitable for this 2. Some data aim at the basic statistic like mean. I think you could do better, having something to compare with these data will make your data look better. For example, average Player Reaction time based on how long they survived. 3. I think this can be more detailed, this can make it easier to implement here is a framework What - what data will you collect Where - where will this data store before you update it into the file (CSV/JSON) When - when will the data be sent to the file (what trigger it, e.g. game end/ each time player press button) Why - why this data is good to have, what does it presenting this will make you visualize how you going to implement it and why you need this Also more specific with the data presentation. If you still do not understand, it is ok because you did not study data science yet. When you study and start to understand how visualization works, revisit and fix it. This is a nice concept and nice game and a very clear plan of implementation. Good job 👍